

TPOT 约束下的 AFD MoE 专家批调度：背景、模型与复杂度

1. 系统背景

Mixture-of-Experts (MoE) 通过为每个 token 选择少量 FFN 专家，在扩大参数规模的同时控制激活计算量 [1]。在自回归 decode 中，一个请求每轮生成一个 token；这个 token 依次经过各层 Attention、路由、专家计算和结果汇合，完成全部层后才能输出。因此，稀疏激活降低了所需计算量，但实际推理效率还取决于这些计算如何在 GPU 上组织。

MoE 的一个主要困难是：**进入模型的 batch 较大，并不意味着每个专家都能获得较大的 batch**。路由会把 token 分散到不同专家，而且分布通常不均匀。冷专家可能反复执行小批次，难以摊薄权重读取开销；热专家则可能成为同步执行中的慢任务，使其他设备等待 [2]。与此同时，Attention 的 KV cache 容量与访问成本限制了可维持的并发量，直接增大整体 batch 还会受到输出时延约束 [3]。

Attention-FFN disaggregation (AFD) 将 Attention 与 FFN 专家部署到不同设备组，使二者可以采用不同的并行方式和资源配置。多个 Attention worker 的输出可以汇入专家侧，为扩大逐专家 batch 提供条件 [3]。在此基础上，如果允许不同请求的专家调用异步排队、重新合批，调度器就需要决定哪些已就绪任务先执行，哪些继续积累。等待可能提高合批效率，也会消耗当前 token 的输出时间预算。本文研究的正是这一等待与启动决策。

2. 现有方法的演进

相关工作主要沿着请求批处理、MoE 执行优化和资源分离三条路线发展，并逐步延伸到模块级异步调度。下面按其解决的问题组织主要工作；这些路线存在并行发展和相互结合。

2.1 从请求批处理到迭代级调度

Orca 将调度粒度从完整请求缩小到一次生成迭代，使已完成请求及时退出、新请求及时加入执行 [4]。vLLM 通过 PagedAttention 减少 KV cache 的碎片与冗余占用，为容纳更多并发请求提供基础 [5]。Sarathi-Serve 进一步切分 prefill，并将其与 decode 组合，缓解长 prefill 对正在生成 token 的请求造成的停顿 [6]。

这些工作分别改善了请求周转、显存利用和 prefill-decode 干扰。对本文而言，还需要继续向模型内部细化调度：同一个生成迭代中的 token 会进入不同专家，整体 batch 的形成不能直接决定每条专家队列的实际 batch 大小与启动时间。

2.2 从专家并行到通信与负载均衡

DeepSpeed-MoE 结合专家并行、专家切分和张量并行，并优化分层 all-to-all 通信及执行 kernel，以支持大规模 MoE 推理 [1]。Lina 进一步分析专家热度偏斜造成的通信不均衡，在推理阶段利用专家选择规律调整资源，改善设备间的传输负载与时延 [2]。

这条路线主要改善专家的分布、资源使用和 token 交换效率。本文关注其后的调度问题：当 placement 和副本绑定已经确定，各队列仍可能以不同速率积累任务，调度器仍需在共享 GPU 上安排执行次序与启动时刻。

2.3 从专家卸载到模块级合批

显存受限场景提供了另一条相关路线。Fiddler 利用 CPU-GPU 协同计算减少专家权重搬运 [7]；MoE-Infinity 利用专家激活轨迹指导缓存替换与预取 [8]；MoE-Lightning 通过 CPU、GPU 与 I/O 的流水重叠和分页权重提高批量推理吞吐 [9]。这些方法使大于显存容量的 MoE 能够更高效地运行。

MoE-Gen 将批处理进一步细化到 Attention 和专家模块：在主存中积累 token，再按模块启动较大的 GPU batch，并联合选择批大小以重叠计算与通信。它面向单 GPU 的离线高吞吐任务，允许用较高时延换取批量效率 [10]。因此，模块级积累与启动在 AMoE 之前已有探索；将这一思路用于交互式 decode 时，需要进一步约束等待对连续 token 输出的影响。

2.4 从 Prefill-Decode 分离到 Attention-FFN 分离

Splitwise 将 prefill 和 decode 放到不同机器，分别配置适合的硬件与资源 [11]。DistServe 同样采用阶段分离，并根据 TTFT、TPOT 要求选择资源与并行配置，以提高满足 SLO 的服务能力 [12]。这类设计缓解两个阶段之间的干扰，分离粒度仍是 prefill 与 decode。

Attention 分离进一步进入一次模型执行的内部。Infinite-LLM 将 Attention 计算与 KV cache 扩展到实例边界之外，以适应动态长上下文 [13]。Adrenaline 将部分 decode Attention 卸载到 prefill 实例，利用两类实例的资源余量 [14]。二者说明，Attention 可以围绕其 KV 状态与资源需求进行独立调度。

面向大规模 MoE，MegaScale-Infer 将 Attention 与专家分离，利用多个 micro-batch 在两侧交替推进的 ping-pong 流水，并通过 M2N 通信降低交换开销。其部署搜索已经包含 SLO 约束，同时选择并行度、Attention 资源数、micro-batch 数和整体 batch 大小 [3]。它建立了高效的分离执行基础；本文进一步关注固定部署之后，异步专家队列上的逐次启动决策。

2.5 AMoE：异步专家并行与重新合批

AMoE 提出 Asynchronous Expert Parallelism (AEP)。任务按层与专家进入独立队列；GPU 可用时选择队列，并用已积累的任务重新合批。同一 GPU 上的多层队列可以交错推进，单个 token 仍须等待其 Top-K 分支汇合。其 defragging scheduler 结合当前队列长度与后续层的加权队列分布，减少批次碎片并改善推进效率 [15]。

这一设计为本文提供了直接的执行背景：专家 batch 的成员可以随实际启动时刻变化。某条队列暂缓执行时，GPU 可以先处理其他队列，而该队列继续积累任务。由此产生的优化空间同时涉及队列选择和启动时机。

2.6 后续相关工作与问题边界

AMoE 之后的 Janus 同样采用 Attention-专家分离，结合激活专家数量的负载均衡、副本与 placement 调整、独立扩缩容来满足 TPOT 要求 [16]。FinDEP 则进一步切分 Attention、专家及通信任务，优化任务粒度、执行次序和流水重叠，并支持 shared expert [17]。

这些工作表明，AFD 下的 SLO 优化和细粒度调度已经受到研究。本文将部署、副本绑定、Attention 与通信响应固定，集中研究异步专家队列的等待与启动。该决策范围与资源扩缩容、专家重放置、计算及通信任务切分有所区别。

3. 现有方法的不足

3.1 TPOT 约束作用于完整输出间隔

交互式 decode 既需要整体吞吐，也需要持续输出。本文采用严格的逐 token TPOT 上界：从同一请求上一 token 的实际输出开始，到当前 token 完整输出为止，整个间隔都必须满足限制。这是本文的建模口径；一些系统使用请求平均 TPOT 或统计 SLO 达成率，不能直接将这些指标等同于逐间隔硬约束。

token 到达某条专家队列时，通常已经消耗了部分预算。此后的等待必须与专家执行、回传、Top-K 汇合、剩余层计算和输出尾部共同分配剩余时间。因此，专家队列不能在任务到达时重新获得一份完整 TPOT 预算，也不能把专家 batch 完成视为该 token 已经输出。即使整体吞吐提高，或平均输出间隔降低，个别 token 仍可能因等待过长而违约。

3.2 队列长度与合批收益不能单独确定等待时间

AMoE 公布的 defragging 评分使用队列数量和层间分布，未将逐 token 的输出截止时刻作为评分输入；其主要性能图报告吞吐与平均 ITL [15]。据此，本文关注一个额外维度：**即使两次决策看到相同的队列长度和合批机会，token 的剩余输出预算不同，也可能要求不同的启动时刻。**

下面用一个示意实例说明。某个末层 Top-1 专家队列当前有 1 个 token，2 ms 后另有 15 个不同请求的 token 到达。GPU 当前空闲且没有其他竞争；容量为 16，大小 1-16 的 batch 服务时间均为 5 ms，服务完成后还需 1 ms 才能输出。后来到达的 token 预算充足。这是用于解释决策差异的给定 profile，不是对所有硬件或服务曲线的假设。

当前 token 的剩余输出预算	立即启动	等待 2 ms 后合批	对当前 token 的影响
6 ms	6 ms 后输出	8 ms 后输出	等待会违约
12 ms	6 ms 后输出	8 ms 后输出	两者均满足预算，等待可减少一次专家执行

立即启动时，后来到达的 15 个 token 需要再执行一批，专家通道共占用 10 ms；等待后可以合成一批，通道只占用 5 ms。两种场景具有相同的服务收益，但只有预算宽松的场景允许这次等待。这个例子中的到达时间用于说明机制；在线调度不能据此假设未来路由已知。

3.3 等待决策必须考虑共享 GPU 与后续依赖

实际系统中，可等待多久还取决于其他任务。一个专家 batch 一旦启动，在本文模型内便不可抢占，因而可能阻塞同 GPU 上即将紧迫的队列。反过来，提前执行紧迫队列也可能为其他队列积累更大 batch 创造时间。若多个专家服务同一 token，提前完成一个分支未必提前最终输出；其他 Top-K 分支或后续层仍可能决定完成时刻。

因此，固定合批阈值、统一等待窗口或只按队列长度排序，都缺少区分上述情况所需的信息。仅按输出期限排序也没有直接确定合法 batch 成员及合批收益。调度需要结合当前可执行任务、profile 给出的服务时间和完整输出路径，协调多条队列的启动。逐 token 检查 SLO，并不意味着不同 token 的调度可以相互独立。

已有方法已经提供了 SLO 约束下的阶段配置、部署搜索和资源管理 [12][3][16]。本文在此基础上研究更具体的问题：**在固定 AFD 部署中，如何根据逐 token 的剩余输出预算选择专家队列和启动时刻，在满足全部 TPOT 上界的前提下利用合批机会，提高既定工作量的完成吞吐？** 后文将这一问题表述为固定输出数下的完工时间最小化；批次成员由实际启动时的队列状态决定，未来路由的可见范围分别按离线与

在线条件限定。

4. 问题定义

在 Attention-FFN 分离的 MoE 推理系统中，Attention workers 完成路由后，将 token 的专家任务发送到对应 FFN GPU。来自不同请求的任务异步到达逐层专家队列，可以重新形成专家批次；同一 token 必须等待本层全部 Top-K 结果返回并完成 combine，才能进入下一层。本文沿用 AMoE 的这一异步执行背景 [15]，研究 decode 阶段的专家队列选择与批次启动。

等待可能使更多任务合批，从而减少总专家服务时间；立即启动则可能缩短当前 token 的等待，但减少后续合批机会。这两种影响还会通过 Top-K 汇合、层间依赖和自回归过程传播。因此，调度需要同时考虑局部合批收益与完整输出路径上的剩余时间预算。

给定固定部署和已接纳的有限 decode 工作量，本文的问题是：**每次选择哪条专家队列、何时实际启动，才能在满足所有逐 token TPOT 上界的条件下，最小化全部输出的完成时间？** 批次成员由启动时的队列状态决定，不作为独立决策。所有必需任务都要执行，不允许通过丢弃请求或减少输出数改善目标。

部署、placement、副本绑定和路由轨迹在一个实例内固定。每张 FFN GPU 上的全部层与专家队列共享一个不可抢占的专家执行通道。Attention/router、发送、回传、combine 和输出尾部采用给定的非负响应延迟。这是独立的建模近似，适用于相应竞争可以忽略或已被资源隔离的条件，不能由“非 FFN 调度策略固定”直接推出。占用专家通道的准备和收尾计入专家服务，其余固定开销计入相应响应延迟，避免重复计费。

5. 十类核心符号

本文的模型只使用以下十类核心符号，包括 token、层、专家和 batch 四种索引。证明中的少量局部量在使用处定义，不加入模型符号体系。

符号	定义	性质
i	一次完整的 token 生成，含全部层及最终输出	固定编号
l	按模型执行顺序排列的 MoE 层（block）编号	固定编号
e	层 l 内的物理专家编号；逻辑身份及副本身份已给定	固定编号
b	一次实际 expert batch； $ b $ 是成员数	由启动动作产生
a_i	同一请求上一 token 的实际输出时刻	首步为边界输入；后续由执行产生
F_i	token i 的最终输出时刻	派生量
s_b	batch b 在所属 GPU 上实际开始服务的时刻	调度决策
c_b	batch b 的服务完成时刻	派生量
$\tau_{l,e}(\cdot)$	第 l 层专家 e 按 batch 大小给出的正服务时间表	固定输入
Δ_i	token i 所属请求的正 TPOT 上限	固定输入；同请求各 token 相同

每条专家队列由二元组 (l, e) 标识，即“第 l 层的物理专家 e ”。例如，队列 $(3, 5)$ 与 $(7, 5)$ 属于不同层，即使专家编号相同，也不能合并；同一 (l, e) 也不能拆成多条独立调度队列。一次专家调用由 token i 及其目标队列确定，它不是一次完整 token 生成。每个 batch 有唯一身份及所选队列记录，即使两个 batch 的 token 成员相同，也不是同一个 batch。

每个调用的实际到达时刻仍由第 6 节的完整计算链精确确定，但不再单独设置带 token、层、专家三重下标的符号。层和专家身份始终显式保留。

请求归属、同请求前驱、逐层路由、GPU 归属、容量和非 FFN 延迟均保留为显式输入记录，不再各自分配符号。token 按输入中的固定请求顺序、再按请求内先后统一编号；这保持原模型的同刻 FIFO 次序。编号本身不表示不同请求之间的执行先后。

6. 模型说明

1. 完整计算链。

a_i 等于同请求前一个 token 的实际输出时刻；窗口内首个 token 使用给定的边界输出。每层先完成 Attention/router 和各分支的发送，输入实际到达对应队列 (l, e) 后，才能参与合批。

专家任务随所在 batch 在 c_b 完成，再分别经过各自的回传延迟。同一 token 等待本层全部 Top-K 结果返回，完成 combine 后进入下一层；末层 combine 及输出尾部完成后产生 F_i ，即服务端采样并提交该 token 的时刻。

所有非 FFN 事件均按给定响应延迟递推。因此，后续 a_i 、实际到达和 F_i 都受完整计算链约束，不能由调度器任意指定或额外推迟。

2. 启动决定合批。

调度器选择队列 (l, e) 和实际启动时刻 s_b 。启动时，从该队列取出容量允许的最长 ready FIFO 前缀，形成 batch b 。队列先按实际到达时刻排序，同刻再按固定 token 编号 i 排序。

同刻完成、到达及其触发的零延迟事件先于启动处理。成员在启动时离队，之后到达的任务不能加入正在执行的 batch；每个所需专家调用恰好执行一次。

3. 服务与资源约束。

对队列 (l, e) 上启动的 batch b ：

$$c_b = s_b + \tau_{l,e}(|b|).$$

s_b 不早于观察原点，且每个成员的完整输入都必须在 s_b 时已经实际到达。batch 在 $[s_b, c_b)$ 内不可抢占；同一 GPU 上所有层、所有专家的 batch 区间不得重叠。GPU 可以主动等待。

4. 逐 token 的 TPOT 约束。

$$F_i - a_i \leq \Delta_i \quad \forall i.$$

每个 token 的输出间隔都满足其所属请求的 TPOT 上限。这个间隔包含队列等待、专家执行、回传、汇合及输出尾部。

5. 优化目标。

将固定观察原点平移到 0，在完成全部既定 token、满足上述约束的前提下，最小化

$$\max_i F_i.$$

固定输出数下，这对应完成吞吐的最大化。

7. 形式化定义

输入： 既定 token i 、专家队列 (l, e) 、服务时间表 $\tau_{l,e}(\cdot)$ 和 TPOT 上限 Δ_i ；同时给定部署、路由、容量、固定响应延迟、请求前驱关系及各请求首步的 a_i 。

输出： 一系列实际启动动作，每项指定 batch b 选择的队列 (l, e) 及启动时刻 s_b 。batch 成员、 c_b 和 F_i 由实际执行产生。

优化目标：

$$\begin{array}{ll} \text{minimize} & \max_i F_i \\ \text{subject to} & F_i - a_i \leq \Delta_i, \quad \forall i. \end{array}$$

“合法”指完成全部必需调用，并遵守前述计算链、FIFO 合批及 GPU 执行规则。后续 token 的 a_i 随前一个 token 的实际输出产生。

8. 复杂度证明

本文只采用一条与等待合批直接相关的 PARTITION 规约。它证明 NP-hardness。

8.1 NP-hardness

定理 1。 本模型的实际完工上界判定是 NP-hard，精确优化也是 NP-hard。困难性已出现在单层、单 GPU、Top-1、每请求仅一次输出、所有非 FFN 响应为 0 的实例中；所有构造实例均满足 SLO 可行，且最优值达到。

证明只新增三个局部量： w_e 为来源整数， A 为总和的一半， S 为某个子集的整数和。

来源问题。 给定一系列二进制正整数 w_e ，总和为 $2A$ ，判断能否选出总和恰为 A 的子集。这是 PARTITION 的 NP-complete 形式 [18]。限制总和为偶数不减弱困难性：把任意正整数 PARTITION 输入的每项都乘 2，即可得到这种形式并保持答案。

构造。 只使用第 1 层。对每项 w_e 创建独立专家 e 及其队列 $(1, e)$ ，所有队列放在同一 GPU。该队列容量为 2，完整服务表为

$$\tau_{1,e}(1) = 2w_e, \quad \tau_{1,e}(2) = 3w_e.$$

为它创建两个属于不同请求的 token，称为早 token 和晚 token，各只生成一次输出，均路由到该队列。另建一条容量为 1、单次服务为 1 的独立队列，放置一个普通的分隔 token。所有专家的逻辑身份不同，所有非 FFN 响应为 0。

token 类型	上一输出边界 a_i	实际 ready	TPOT 上限 Δ_i	由 TPOT 得到的最终输出 上限
每条队列的早 token	0	0	$8A + 1$	$8A + 1$
每条队列的晚 token	$2A + 1$	$2A + 1$	$6A$	$8A + 1$
分隔 token	$2A$	$2A$	1	$2A + 1$

设置公共完工上界为 $7A + 1$ 。全部编号按来源顺序固定，不依赖未知子集。晚到达由合法的将来边界输出产生，不是新增的 release 决策；两个 item token 不是同请求的连续输出。合批有实际服务收益，因为 $3w_e < 4w_e$ 。

任何来源输入都有 SLO 可行日程。 GPU 等待到 $2A$ ，执行分隔 token 至 $2A + 1$ ；随后逐队列执行 size-2 batch。后段总服务为 $6A$ ，最后输出在 $8A + 1$ ，所有 TPOT 均满足。该日程不依赖 PARTITION 的答案，因此困难性不来自先判断 SLO 可行域是否为空。

分隔区间。 在任意 SLO 可行日程中，分隔 token 不早于 $2A$ ready，服务为 1，又必须不晚于 $2A + 1$ 输出，故恰好占用 $[2A, 2A + 1)$ 。其他 batch 不可横跨这一区间。分隔前能执行的 item batch 只能是早 token 的 singleton。

正向。 若存在总和为 A 的子集，先执行这些队列的早 token，总服务为 $2A$ ；接着执行分隔 token。时刻 $2A + 1$ ，先前启动过的队列仅剩晚 token，须执行 singleton；其余队列有早、晚两个成员，full-drain 强制形成 size-2。后段服务为

$$2A + 3(2A - A) = 5A.$$

所以全部输出在 $7A + 1$ 完成，且满足所有 TPOT，得到目标判定的 YES。

反向。 取任意达到公共完工上界的合法日程，将分隔前已完成早 token 的队列对应整数相加，记为 S 。这些 singleton 必须在长度 $2A$ 的前段内完成，因此

$$2S \leq 2A, \quad S \leq A.$$

其他 item 不可能处在跨越分隔区间的 batch 中。到 $2A + 1$ ，已经提前执行的队列只剩晚 token，其余队列的两个成员均在等待。按 full-drain，后段必要服务恰为

$$2S + 3(2A - S) = 6A - S.$$

允许任何合法次序及额外空闲，仍有

$$\max_i F_i \geq (2A + 1) + (6A - S) = 8A + 1 - S.$$

结合 $\max_i F_i \leq 7A + 1$ ，得到 $S \geq A$ ，故 $S = A$ ，恢复 PARTITION 的解。反向使用任意合法日程，不要求它预先符合正向构造的顺序。

规模与最优可达性。 每个来源整数只产生两 token、一条队列和两个 profile 表项，另加一个分隔 token；构造数值是输入整数及其总和的常数倍，二进制长度为多项式。

更一般地，任一整数和为 $S \leq A$ 的队列子集，都可先用 $2S$ 时间执行其早 singleton，空闲至 $2A$ ，执行分隔 token，然后无空闲地完成其余工作。其完工时间恰为 $8A + 1 - S$ ，并满足 SLO。前述下界覆盖所有合法日程，而这样的子集只有有限个，故取其中最大的可行子集和即可实际达到本构造的最优值。因此，即使承诺 SLO 可行且最优值达到，精确优化也能通过与 $7A + 1$ 比较解出 PARTITION，仍为 NP-hard。证毕。

8.2 任意固定 Top-K

定理 1 并不要求实际研究采用 Top-1。对任意预先固定的 Top-K 设置，使用与选中专家数相同数量的 GPU；在每张 GPU 上，为每项整数和分隔 token 各建一个不同的逻辑专家。每个 token 路由到各 GPU 上对应的专家，逐 GPU 复制原容量与服务表，边界和 SLO 不变。

正向在所有 GPU 同步复制上述日程，完整输出时刻不变。反向不假设同步：分隔 token 必须等全部分支完成，因此每张 GPU 的分隔服务都被迫占用 $[2A, 2A + 1)$ 。任取一张 GPU，其每个分支完成不晚于所属 token 的最终输出；在这张 GPU 上重复前述服务量下界，即可提取总和为 A 的子集。不同 GPU 可以采用不同的执行顺序。

相同论证也保持所有构造实例的 SLO 可行性。每张 GPU 的下界都由不超过 A 的子集和给出；选择最大可行子集和并同步复制达到该下界，所以扩展后的最优值也达到。Top-K 固定时，复制仅带来常数倍规模。

参考文献

- [1] Samyam Rajbhandari et al. *DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale*. ICML, 2022. 推理并行、分层 all-to-all 与 kernel 优化见 §5。
[DeepSpeed-MoE](#)
- [2] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, Hong Xu. *Accelerating Distributed MoE Training and Inference with Lina*. USENIX ATC, 2023. 此处引用其推理阶段的专家负载与通信优化。[Lina](#)
- [3] Ruidong Zhu et al. *MegaScale-Infer: Serving Mixture-of-Experts at Scale with Disaggregated Expert Parallelism*. arXiv:2504.02263v1, 2025. 分离与 ping-pong 流水见 §3-§4.1；含 SLO 的部署搜索见 §4.2。[MegaScale-Infer](#)
- [4] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, Byung-Gon Chun. *Orca: A Distributed Serving System for Transformer-Based Generative Models*. OSDI, 2022. [Orca](#)
- [5] Woosuk Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP, 2023. [vLLM / PagedAttention](#)
- [6] Amey Agrawal et al. *Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve*. OSDI, 2024. [Sarathi-Serve](#)
- [7] Keisuke Kamahori, Yile Gu, Kan Zhu, Baris Kasikci. *Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models*. arXiv:2402.07033v1, 2024. [Fiddler](#)
- [8] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, Mahesh Marina. *MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache*. arXiv:2401.14361v3, 2025；首稿发布于 2024 年。[MoE-Infinity](#)
- [9] Shiyi Cao et al. *MoE-Lightning: High-Throughput MoE Inference on Memory-constrained GPUs*. arXiv:2411.11217, 2024. [MoE-Lightning](#)
- [10] Tairan Xu, Leyang Xue, Zhan Lu, Adrian Jackson, Luo Mai. *MoE-Gen: High-Throughput MoE Inference on a Single GPU with Module-Based Batching*. arXiv:2503.09716v1, 2025. 离线吞吐任务定位见 §1，模块级批处理见 §4。[MoE-Gen](#)
- [11] Pratyush Patel et al. *Splitwise: Efficient Generative LLM Inference Using Phase Splitting*. ISCA, 2024. [Splitwise](#)
- [12] Yinmin Zhong et al. *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving*. OSDI, 2024. [DistServe](#)
- [13] Bin Lin et al. *Infinite-LLM: Efficient LLM Service for Long Context with DistAttention and Distributed KVCache*. arXiv:2401.02669v2, 2024. [Infinite-LLM](#)
- [14] Yunkai Liang, Zhangyu Chen, Pengfei Zuo, Zhi Zhou, Xu Chen, Zhou Yu. *Injecting Adrenaline into LLM Serving: Boosting Resource Utilization and Throughput via Attention Disaggregation*. arXiv:2503.20552, 2025. [Adrenaline](#)
- [15] Shaoyu Wang, Guangrong He, Geon-Woo Kim, Yanqi Zhou, Seo Jin Park. *Toward Cost-Efficient Serving of Mixture-of-Experts with Asynchrony*. arXiv:2505.08944v2, 2025. 异步队列、token 元数据、重新合批和 Top-K 依赖见 §3.1-§3.2；评分见 §3.4 的 Algorithm 1；平均 ITL 评估见 §5.1。本文的硬 TPOT、FIFO、容量截断和固定响应是另外明确的建模约定。[AMoE](#)
- [16] Zhexiang Zhang et al. *Janus: Disaggregating Attention and Experts for Scalable MoE Inference*. arXiv:2512.13525v1, 2025. 激活负载调度见 §3.3；专家管理和扩缩容见 §3.4。[Janus](#)

[17] Xinglin Pan et al. *Efficient MoE Inference with Fine-Grained Scheduling of Disaggregated Expert Parallelism*. arXiv:2512.21487v1, 2025. 任务切分与排序见 §3-§4; 在线配置调整见 §5.5。 [FinDEP](#)

[18] Richard M. Karp. *Reducibility among Combinatorial Problems*. In *Complexity of Computer Computations*, pp. 85–103, 1972. 本文使用其 PARTITION 问题的正整数、二进制编码版本。 [出版页面](#)